

# Mil Ojos

Metodología técnica del exoesqueleto de nueve cámaras



---

**AUTOR**

Manuel Alejandro Mendoza Valencia

**VERSIÓN DEL SISTEMA**

v5: 9 brazos × 3 servos, 3 controladores PCA9685

**INTERFAZ WEB**

milojos.totot.mx

**PERIODO REPORTADO**

Junio a septiembre de 2026

**CÓDIGO**

[github.com/manuTototDev/mil-ojos](https://github.com/manuTototDev/mil-ojos)

---

## 01 Premisa

Nadie puede reconocer, retener y cotejar los cientos de rostros de personas desaparecidas que circulan a diario en afiches, pantallas y redes. Un rostro visto tres segundos en una ficha de búsqueda se pierde en la siguiente esquina. La falla no es moral, es fisiológica.

*Mil Ojos* es una prótesis para ese límite: un exoesqueleto que se lleva en la cabeza, con nueve cámaras que buscan rostros en la multitud y los comparan en tiempo real con registros públicos de personas desaparecidas. Este documento describe cómo está construido y cómo funciona: el hardware, la calibración, la visión computarizada, el control de los motores y las decisiones técnicas que se tomaron en el camino.

**9**

brazos con cámara

**27**

servomotores

**3**

controladores PWM (I<sup>2</sup>C)

**11,375**

rostros indexados en la base local

---

## 02 Arquitectura general

El sistema tiene tres capas: **percepción** (cámaras y modelo de rostros), **decisión** (un programa en Python que corre en una computadora) y **actuación** (un microcontrolador Arduino que mueve los 27 servos a través de tres placas PCA9685). La computadora y el Arduino se comunican por puerto serial.

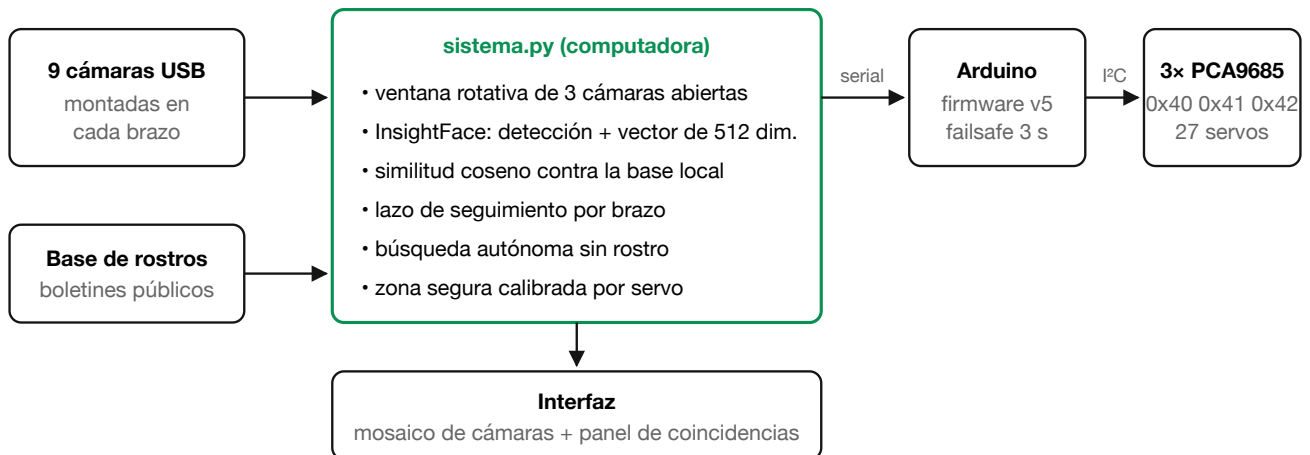


Figura 1. Flujo de datos: captura → detección y cotejo → decisión de movimiento → servos.

---

## 03 Hardware

| Componente       | Detalle                                                                                                                                                            |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Brazos           | 9 brazos articulados de 3 grados de libertad: <b>base</b> (giro horizontal), <b>hombro</b> y <b>codo</b> (inclinación). Cada brazo lleva una cámara en el extremo. |
| Servomotores     | 27 servos, controlados con pulsos PWM a 50 Hz (rango de pulso 150-600 ticks).                                                                                      |
| Controladores    | 3 placas PCA9685 de 16 canales en cadena I <sup>2</sup> C, con direcciones 0x40, 0x41 y 0x42. Cada placa maneja 3 brazos (9 canales).                              |
| Microcontrolador | Arduino conectado por USB serial a 115200 baudios.                                                                                                                 |
| Cámaras          | 9 webcams USB idénticas, montadas de lado: la imagen se rota 90° en software.                                                                                      |
| Estructura       | Bastidor de madera atornillado que sostiene los brazos alrededor de la cabeza. Cableado de datos con conectores modulares para poder dar mantenimiento por brazo.  |
| Procesamiento    | Computadora con el modelo de IA corriendo en CPU.                                                                                                                  |

---

## 04 Calibración de los brazos

Cada servo tiene un rango físico distinto según cómo quedó montado. Mandar un ángulo fuera de ese rango hace que el motor fuerce contra la estructura. Por eso cada brazo se calibra uno por uno con `calibrar_brazos.py` y un firmware dedicado (`calibrador.ino`):

- **Maapeo canal → servo.** Cada canal se mueve en un barrido lento para identificar qué articulación responde.
- **Zona segura.** Para cada servo se fijan a mano un mínimo, un máximo y una posición de reposo (*home*).
- **Servos invertidos.** Los que quedaron montados al revés se marcan como invertidos y el software les da la vuelta, de modo que todo el sistema habla en *ángulos lógicos* con la misma convención.

El resultado es un archivo JSON por brazo. `generar_header.py` convierte esos archivos en `config_brazos.h`, que se compila dentro del firmware. Así **la zona segura se aplica en el propio Arduino**: aunque el programa de Python falle o alguien escriba comandos a mano, ningún servo puede salir de su rango.

---

## 05 Emparejamiento cámara ↔ brazo

Las nueve cámaras son idénticas y no tienen número de serie, así que el sistema operativo no las distingue, y su orden cambia en cada arranque. Para saber qué cámara pertenece a qué brazo, `emparejar_cameras.py` sacude cada brazo y mide cuánto se desplaza la imagen de cada cámara usando **correlación de fase**:

- La cámara montada en el brazo que se mueve ve desplazarse **todo su encuadre**, alrededor de 12 px.
- Las demás cámaras apenas registran cambio, menos de 1 px.

La decisión es relativa: cada cámara se compara consigo misma entre brazos, así que no depende de la iluminación ni del contraste de la escena. El mapeo resultante se guarda como un archivo de sesión y se regenera en cada arranque.

---

## 06 Percepción y cotejo de rostros

La detección y el reconocimiento usan **InsightFace** con el modelo `buffalo_l`. Para cada cuadro de video:

1. Se detectan los rostros; se elige el de mayor tamaño (el más cercano).
2. Se calcula su **vector de 512 dimensiones** (embedding normalizado).
3. Se compara contra todos los vectores de la base con **similitud coseno**, que es un solo producto matricial, en milisegundos.
4. **Votación temporal.** El candidato más parecido se guarda en un historial de los últimos 15 cuadros, y lo que se muestra solo cambia cuando un nuevo candidato aparece al menos 8 veces. Esto evita que el resultado salte de una persona a otra con cada cuadro.
5. La interfaz muestra las **8 coincidencias más altas** con su porcentaje de similitud.

**Base de datos.** La base se construye localmente a partir de boletines públicos de búsqueda de la Comisión de Búsqueda de Personas del Estado de México (2020-2026) y del Registro Nacional de Personas Desaparecidas y No Localizadas. El proceso descarga los boletines, recorta y normaliza la fotografía y la indexa en un archivo local. Hoy la base tiene 11,375 rostros.

**Especificación honesta.** La similitud máxima observada entre un rostro en vivo y la base ronda el 30-36 %. El sistema no confirma identidades ni pretende hacerlo: muestra parecidos. La obra no está en el acierto de la máquina, sino en su insistencia.

---

## 07 Manejo de nueve cámaras en tiempo real

El bus USB 2.0 no tiene ancho de banda para mantener las nueve cámaras transmitiendo al mismo tiempo. La solución fue una **ventana rotativa**:

- Hay como máximo **3 cámaras abiertas** a la vez. Cada 8 s se cierra la que lleva más tiempo abierta y se abre la siguiente en turno.
- **La rotación respeta a quien está mirando.** Una cámara que vio un rostro en los últimos 4 s no se cierra aunque le toque.
- Cada cámara captura en su propio hilo, en formato MJPG con un búfer de un solo cuadro, para no acumular retraso.
- La detección se reparte por turnos (una cámara por vuelta del ciclo) para no saturar el procesador. Además, el tamaño de detección se redujo a 320 × 320 px para ganar cuadros por segundo.

- Un brazo cuya cámara está cerrada **no se congela**: sigue buscando con su movimiento autónomo.

---

## 08 Control de movimiento

### Seguimiento de rostro (lazo cerrado por brazo)

Cuando una cámara detecta un rostro, su brazo intenta llevarlo al centro de la imagen. Se calcula el error entre el centro del rostro y el centro del cuadro, se normaliza y se convierte en un paso de hasta  $\pm 5^\circ$  por ciclo:

```
base  (giro)      ← error horizontal
codo  (inclinación) ← error vertical
hombro           → se adelanta 12° sobre su reposo, para "asomarse" hacia el rostro
cada valor se recorta a la zona segura del servo
```

### Suavizado

Los motores no saltan al ángulo nuevo. En cada ciclo avanzan una fracción de la distancia que les falta (interpolación exponencial): 22 % cuando siguen un rostro y 10 % cuando buscan. El firmware vuelve a suavizar del lado del Arduino. Así el movimiento es continuo, sin tirones, y la cámara no tiembla mientras observa.

### Búsqueda autónoma

Sin rostro, cada brazo se mueve con **ruido suave** (senos superpuestos que aproximan ruido Perlin) dentro del 55 % de su zona segura. Cada brazo tiene su propia semilla, así que los nueve buscan de forma independiente y nunca sincronizada, como una cabeza llena de ojos que miran a todos lados.

### Protocolo serial y seguridad

| Trama / comando                        | Función                                                                                                                      |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>\$a0,a1,...,a26,1</code>         | Ángulos lógicos de los 27 servos, en orden global (brazo $\times$ 3 + articulación).                                         |
| <code>H · X · ?</code>                 | Todos a reposo · liberar servos · estado.                                                                                    |
| <code>PING · SCAN · M · R · RUN</code> | Modo prueba para el emparejamiento de cámaras (mover un servo, liberar, volver a operación).                                 |
| Failsafe                               | Si no llega nada por serial en 3 s, el firmware pasa solo a búsqueda suave. Del modo prueba sale solo tras 5 s sin comandos. |

---

## 09 Interfaz

Dos ventanas para operar la pieza y para el registro de las sesiones:

- **Mosaico de cámaras.** Las cámaras abiertas, etiquetadas por brazo, con cuadros por segundo, marco sobre el rostro detectado e indicador de actividad.
- **Panel de coincidencias.** Las 8 fichas más parecidas, con porcentaje y barra de similitud, el número de personas indexadas y qué brazos esperan turno de cámara.

### Extensión web: [milojos.totot.mx](http://milojos.totot.mx)

La interfaz de visualización también existe como sitio web público, [milojos.totot.mx](http://milojos.totot.mx), para que cualquier persona use la cámara de su propio dispositivo como un ojo más del sistema. Tiene tres vistas:

- **Escaneo:** captura el rostro del visitante y muestra en tiempo real las fichas más parecidas, con su porcentaje de similitud y los puntos faciales detectados.
- **Explorar:** galería paginada de las fichas de búsqueda, con filtros por año.
- **Ficha:** detalle de cada persona, con su boletín original.

El frontend está hecho en Next.js 15 y React 19 y se publica en Vercel. La búsqueda la hace una API en FastAPI con el mismo modelo, InsightFace `buffalo_l`, alojada en Hugging Face Spaces; compara el vector del rostro contra toda la base en menos de 100 ms.

## 10 Desarrollo por etapas

| Periodo    | Meta comprometida                                                  | Resultado técnico                                                                                                                                                               |
|------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ene-may    | Investigación, diseño, prototipo, control de cámaras, módulo de IA | <b>v3:</b> una placa de 16 canales, un brazo de seguimiento con una cámara y tres brazos acompañantes; módulo de IA y base de rostros funcionando.                              |
| Junio      | Sistema cámara-IA-motor con 5 cámaras                              | La salida de la IA se conecta al control de servos; el flujo captura → IA → cotejo → movimiento se prueba con 5 cámaras; se ajusta la velocidad de respuesta.                   |
| Julio      | 9 cámaras operativas; interfaz para prueba pública                 | Ventana rotativa para las 9 fuentes; emparejamiento automático cámara-brazo; optimización de la IA (detección a 320 px, captura en hilos); interfaz de mosaico y coincidencias. |
| Agosto     | 3 sesiones de prueba; informe de ajustes                           | Calibración real de cada brazo y zona segura en firmware; prueba continua de 4 h; sesiones de prueba documentadas.                                                              |
| Septiembre | Exoesqueleto operativo; carpeta de registro                        | <b>v5:</b> 9 brazos × 3 servos sobre 3 placas; ensamble y cableado definitivos; documentación técnica y código organizados.                                                     |

---

## 11 Datos y ética

- La base de rostros se construye solo con **boletines públicos** de búsqueda. El exoesqueleto la usa de forma local.
- La extensión web muestra las fichas tal como las publica la comisión de búsqueda, con su boletín original.
- El repositorio de código no contiene fotografías, nombres, fichas, bases de datos ni registros de personas.
- El sistema guarda capturas solo de forma local, como registro de las sesiones.
- La pieza no identifica a nadie: muestra parecidos con un porcentaje explícito. Su función es poética y política: sostener la mirada sobre las personas ausentes que el ojo humano no alcanza a retener.

---

## 12 Código fuente

Repositorio: [github.com/manuTototDev/mil-ojos](https://github.com/manuTototDev/mil-ojos) · Interfaz web: [milojos.totot.mx](https://milojos.totot.mx)

| Archivo                           | Función                                          |
|-----------------------------------|--------------------------------------------------|
| python/sistema.py                 | Sistema completo v5                              |
| python/calibrar_brazos.py         | Calibración por brazo                            |
| python/emparejar_camaras.py       | Emparejamiento cámara ↔ brazo                    |
| python/generar_header.py          | Genera la configuración del firmware             |
| arduino/milojos/milojos.ino       | Firmware principal (27 canales, 3 placas)        |
| arduino/calibrador/calibrador.ino | Firmware de calibración                          |
| web/frontend/                     | Interfaz web de visualización (milojos.totot.mx) |
| web/backend/main.py               | API de búsqueda para la interfaz web             |

## 13 Registro fotográfico

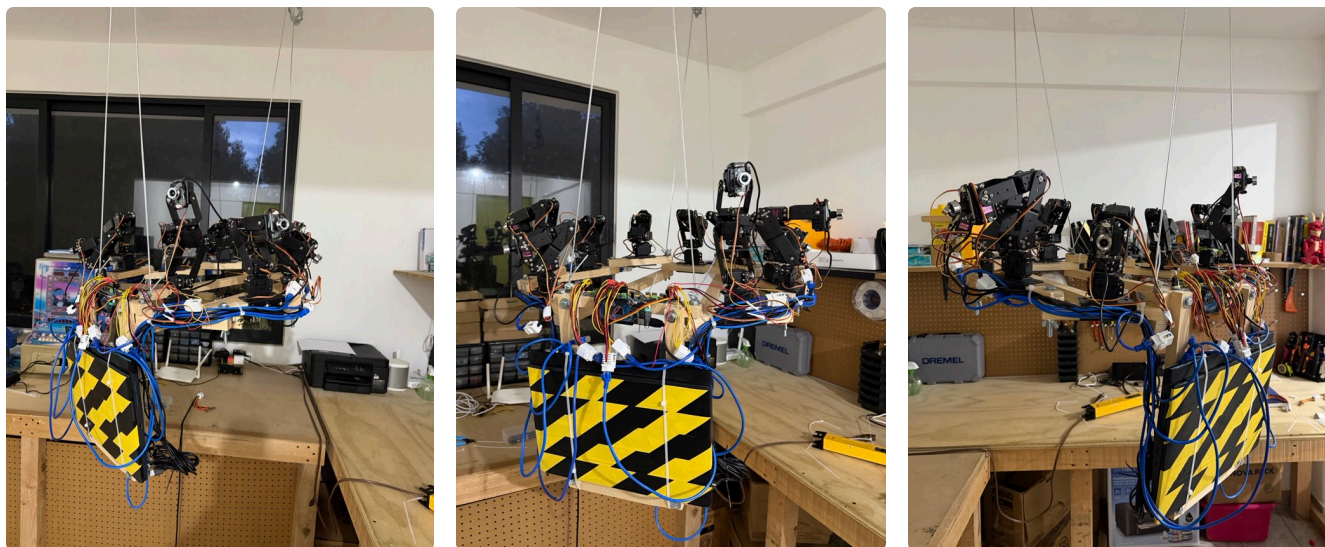


Figura 2. Exoesqueleto en el taller, en su soporte de suspensión: brazos con cámara, controladores y cableado.